APPLICATION LAYER PROTOCOLS (HTTP)

CIS*3210

Fall 2026

Sajid Marhon

Based on notes for Computer Networking: A Top-Down Approach, 9th ed., by J. Kurose and K. Ross

Learning Outcomes



By the end of this lesson, students will be able to:

- Identify common Internet applications and their architectures
- Explain process communication using sockets, IP addresses, and ports, and compare TCP and UDP services
- Describe how HTTP works and its evolution from HTTP/1.1 to HTTP/3
- Explain how email and DNS support Internet applications

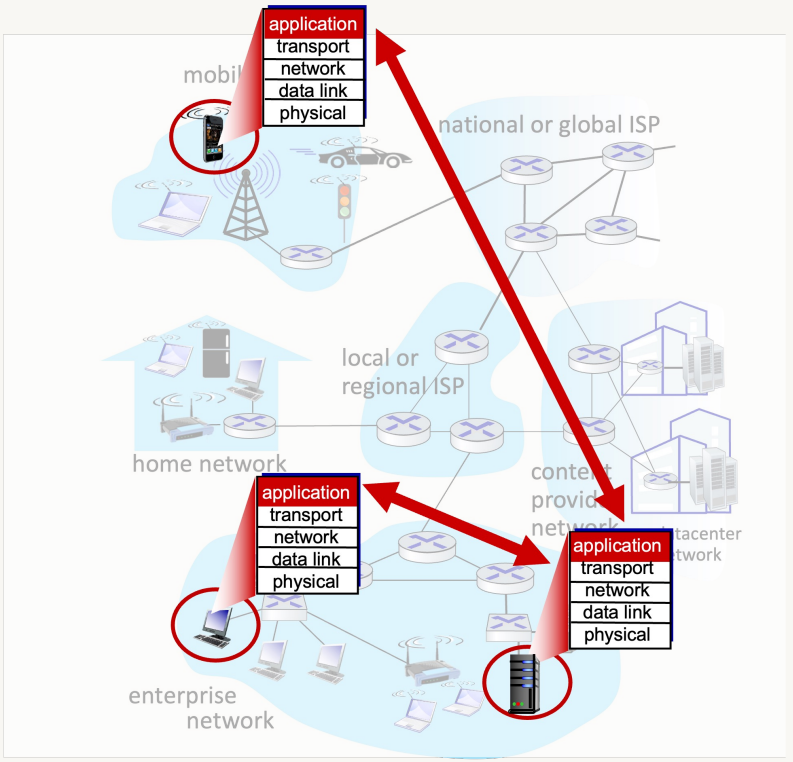
Some Network Applications



- social media
- Web
- text messaging
- e-mail
- multi-user network games
- streaming stored video (YouTube, Hulu, Netflix)
- P2P file sharing
- voice over IP
- real-time video conferencing (e.g., Zoom)
- Internet search
- remote login
- …

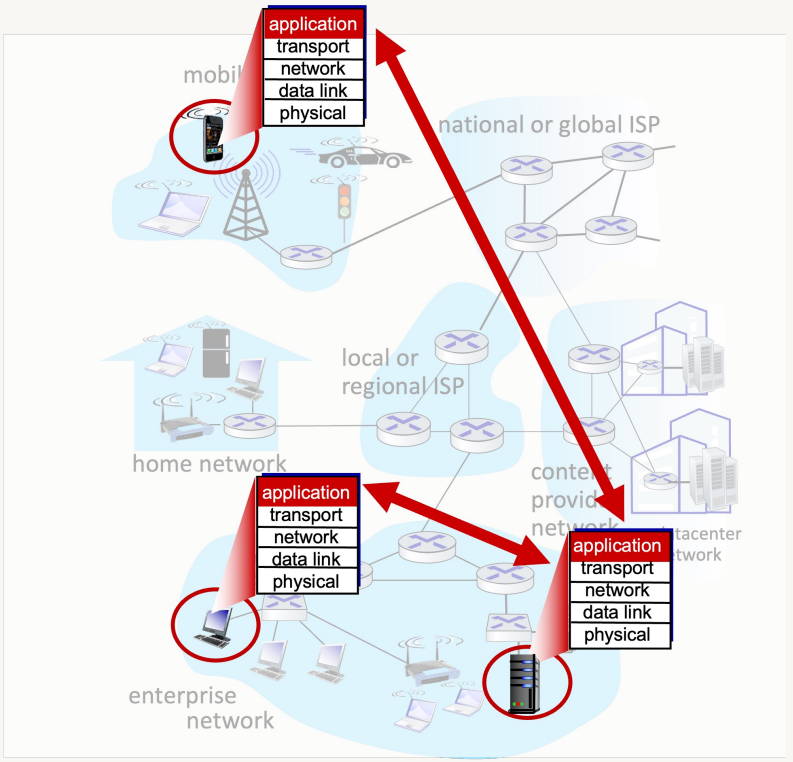
Network Applications

- Network applications typically involve two or more programs exchanging messages
- Web: a browser communicates with a web server
- Video on demand (e.g., Netflix): an app communicates with a streaming server
- User programs run on computers, mobile devices, and smart TVs
- Server programs often run in data centers
- Motivations include public benefit, coursework, profit, or fun



Network Applications

- Implement application logic on end hosts (clients/servers)
  - Programs on different hosts exchange messages
- Example: a browser communicates with a web server via HTTP
- No need to program routers or switches
  - Core devices forward packets; applications run at the edge
  - Edge deployment simplifies development, distribution, and updates



Network Applications



Discussion Question:

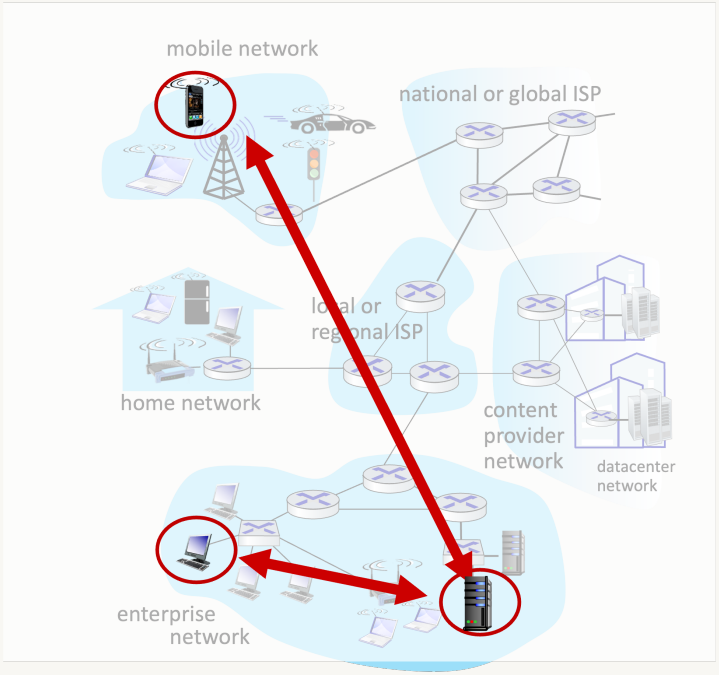
Why can an application be updated globally without changing the forwarding logic in the routers along its paths?

The application logic executes at the edge, while routers implement general packet forwarding. As long as the updated application continues to use supported transport and network services, routers can carry its packets without knowing that the application changed.

Client-Server Architecture



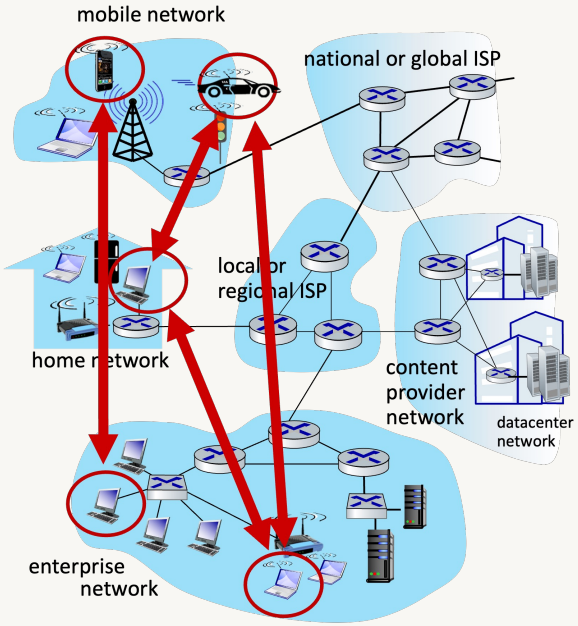
- **Server**
  - Always-on server handles requests from many clients
  - Server has a fixed, well-known IP address, so clients can always reach it
  - Commonly deployed in data centers for scalability and capacity
- **Clients:**
  - Initiate contact and exchange data with the server
  - Can be occasionally online
  - Often receive changing (dynamic) IP addresses
  - Do not communicate directly with other clients



Peer-to-Peer Architecture



- **No always-on central server**
- **Direct communication** between peers
- **Dual roles:** peers request and provide resources
- **Self-scaling:** new peers add capacity and demand
- **User devices:** typically personal computers
- **Intermittent availability:** peers disconnect and IP addresses change
- **Harder management** due to decentralization
- **Lower costs:** reduced server and bandwidth needs
- **Example:** BitTorrent file sharing



Processes Communicating

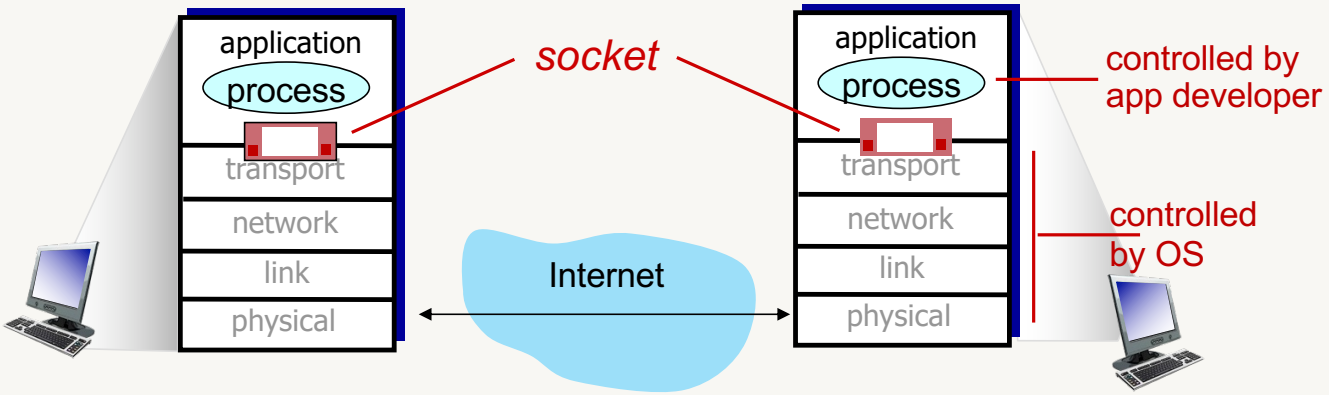


- Programs run as **processes**; processes communicate
- On the same host, processes communicate through the local OS
- Across hosts, processes exchange network messages
- Application processes operate at the **application layer**
- Example: browser and web server processes exchange messages
- **Client process** initiates contact; **server process** waits
- These form the application's **client side** and **server side**
- **P2P applications** also have client and server processes

Sockets



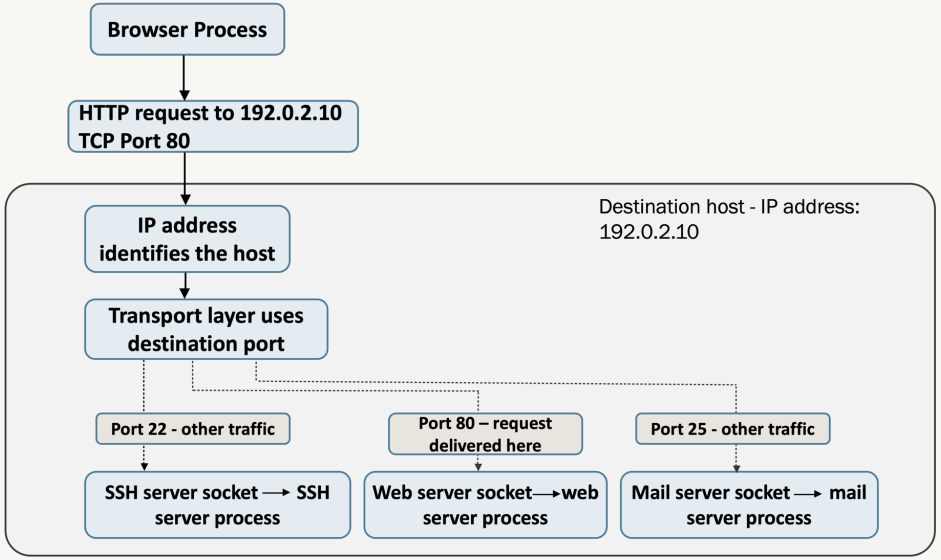
- A socket is the gateway between a process and the network
- A process sends and receives messages through its socket
- **Analogy:** process ≡ house; socket ≡ door, messages exit/enter through the socket
- The transport/network stack carries the message to the peer's socket for delivery
- Each exchange involves two sockets, one at each communicating host



Addressing Processes



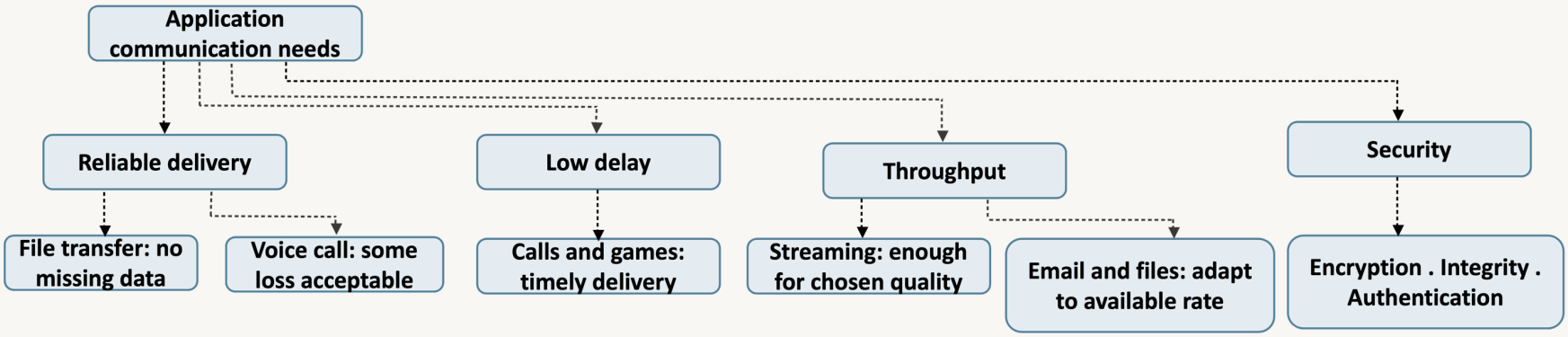
- A receiving process needs a unique identifier to get messages
- A host interface may use a 32-bit IPv4 address or 128-bit IPv6 address
- Would IP address of the host be enough to identify a process?
  - IP alone isn't enough to pinpoint a process on a host
- To reach a specific process, you need two identifiers: the host's IP address and the process's port number
  - A port number identifies the target process/socket on that host (since a host can run many apps)
- Many apps use well-known ports (e.g., HTTP 80, SMTP 25)
- Bottom line: IP = which host, Port = which process



Transport Services for Applications



- **What services can transport protocols offer applications?**
- **Reliable delivery:** requirements vary by application
  - **Loss-intolerant:** file transfers and web transactions need reliable delivery
  - **Loss-tolerant:** real-time media can tolerate some data loss
- **Timing:** Internet calls and interactive games need low end-to-end delay
- **Throughput:** required data rates vary
  - **Minimum-rate apps:** need sufficient bandwidth for a target quality
  - **Elastic apps:** adapt to available bandwidth (e.g., email, file transfers)
- **Security:** encryption, integrity protection, and authentication



Transport Protocol Services



TCP Service

- Reliable, connection-oriented delivery
- Provides flow and congestion control
- Requires a connection setup before data transfer
- No guarantees for latency or minimum bandwidth
- No built-in security

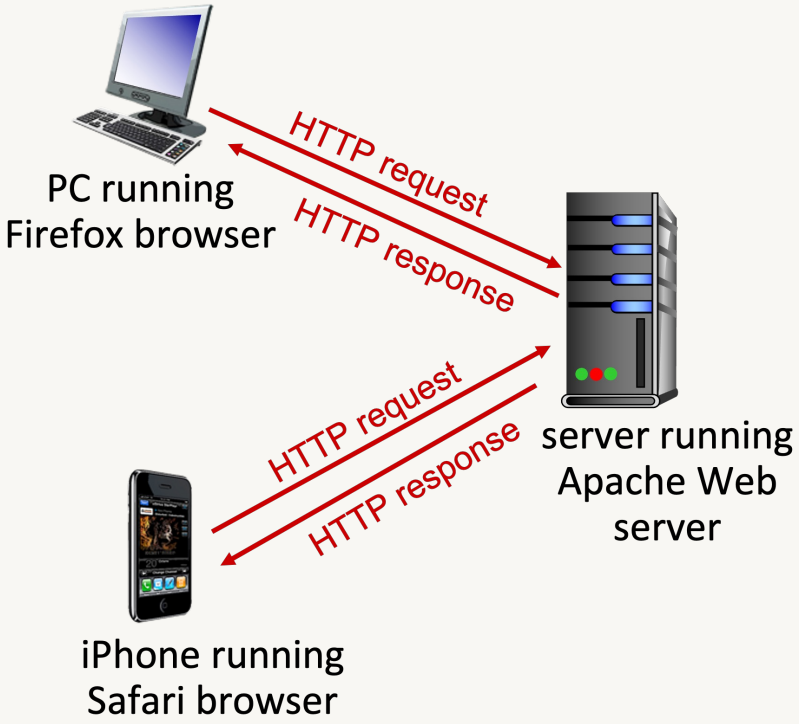
UDP Service

- Best-effort, connectionless delivery
- No reliability, flow, or congestion control
- No latency or throughput guarantees
- No built-in security
- No connection setup required
- **Question:** Why would an application choose UDP?

HTTP



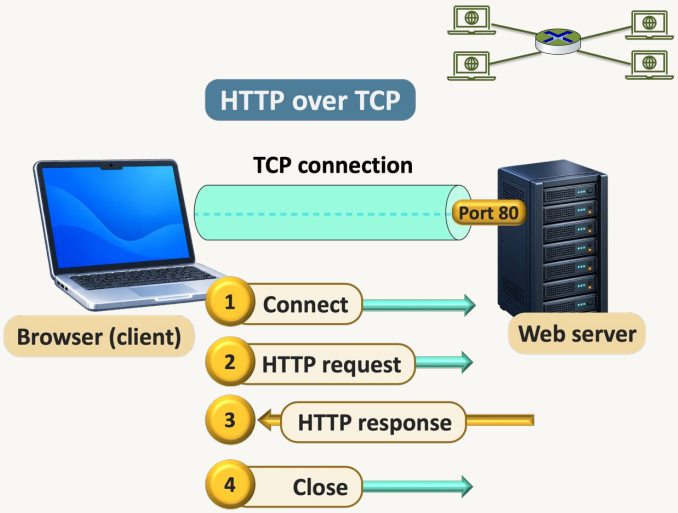
- **HTTP (Hypertext Transfer Protocol)**
- The Web's application-layer protocol
- Uses a client–server architecture
  - **Client:** browser requests, receives, and displays web objects
  - **Server:** delivers the requested objects
- Supports websites, streaming, social media, webmail, and mobile apps
- Uses a request–response model: the client sends a request, and the server returns a response

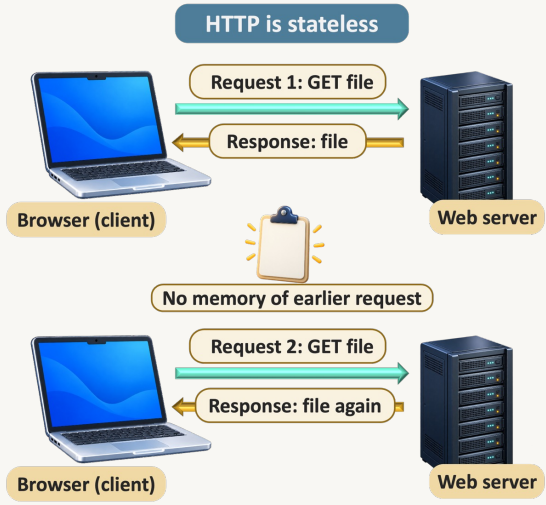


HTTP

- **HTTP/1.1 and HTTP/2 run over TCP**
  - The browser opens a TCP connection to the server, usually on port 80 for plain HTTP; port 443 for HTTPS
  - The server accepts the connection
  - HTTP requests and responses are exchanged over it
  - The connection is closed when no longer needed

- **HTTP is stateless**
  - HTTP does not require the server to retain client state between requests
  - Each request is handled independently

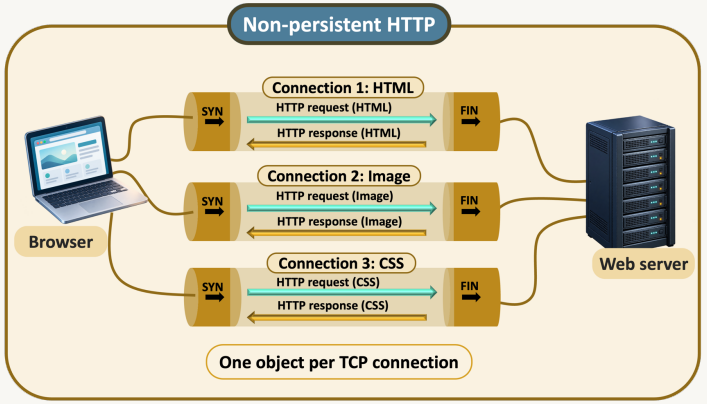


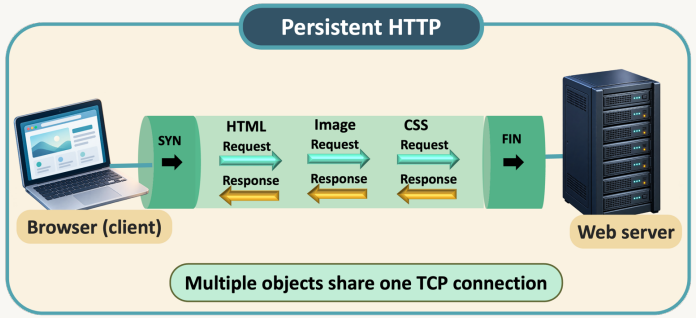


HTTP Connections



- **Connection strategies**
  - **Non-persistent:** each request/response uses a separate TCP connection
  - **Persistent**: multiple requests/responses share one TCP connection
- **Non-persistent HTTP**
  - Establish a TCP connection, transfer one object, then close the connection
  - Each additional object triggers a new TCP connection
  - Result: pages with many objects require multiple separate TCP handshakes and connections
- **Persistent HTTP**
  1. Open a single TCP connection to the server
  2. Reuse that connection to fetch multiple objects between the same client and server (can be pipelined)
  3. Close the connection when done or after an idle timeout





Non-Persistent HTTP: Example



User enters URL: **`www.someSchool.edu/someDepartment/index.html`**
(containing text, references to 10 JPEG images)



1a. Client Initiation: Client initiates a TCP connection to the server (port 80); sockets are created on both ends



1b. The HTTP server on www.someSchool.edu listens on **TCP port 80**; when a client connects, it **accepts** the connection and signals readiness to the client

2. Send request: Client issues an HTTP request (including the object's path) via its socket

3. Server responds: Server reads the request, retrieves the object, wraps it in an HTTP response, and sends it back

4. Close connection: Server requests TCP teardown; TCP ensures delivery before terminating

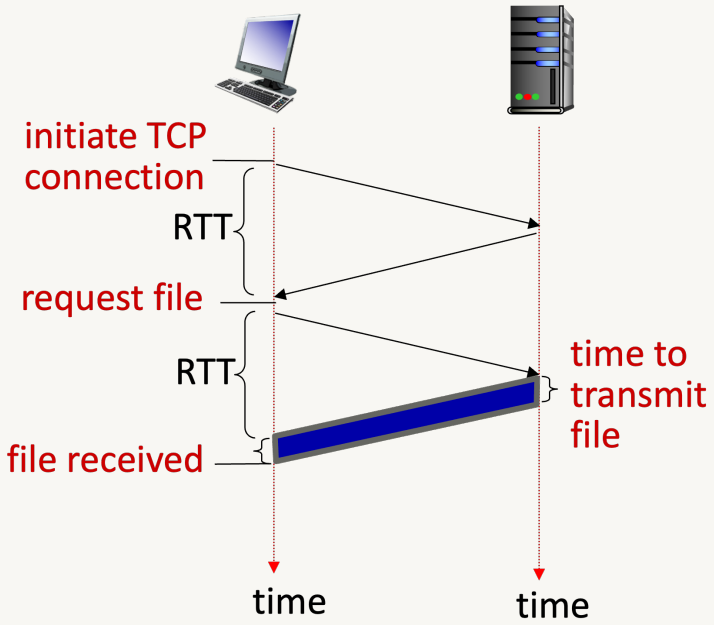
5. Process response: Client receives the response, closes the TCP connection, parses the HTML, and discovers referenced images

6. Repeat per object: Steps 1-5 are repeated for each referenced JPEG (non-persistent behavior)

Non-Persistent HTTP: Response Time



- **RTT (round-trip time):** time for a small packet to go client ⇒ server ⇒ client; includes propagation, queuing, and processing delays
- **HTTP response time (per object):**
  - **TCP setup**: clicking a link triggers a TCP three-way handshake; the first two steps cost 1 RTT
  - **Request send**: client piggybacks the HTTP request with the third ACK
  - **Server reply**: server sends the HTML file; the request/response exchange costs 1 more RTT
  - Total page fetch time (for base HTML): 2 RTTs + server transmission time for the file



Persistent HTTP



- **Non-persistent drawbacks:** each object requires a new TCP connection ⇒ extra setup/teardown, more TCP state/buffers on client & server, and **2 RTTs per object**
- **Persistent HTTP (e.g., HTTP/1.1, HTTP/2):** server keeps the TCP connection open after a response
- **Connection reuse:** multiple requests/responses between the same client and server share one TCP connection
- **Whole page over one connection:** base HTML + all referenced objects can be fetched on a single persistent connection
- **Multiple pages:** even different pages on the same server can reuse the same connection
- **Pipelining (HTTP/1.1):** client can issue back-to-back requests without waiting for prior replies; server returns objects back-to-back
- **Idle timeout:** server typically closes a persistent connection after a configurable period of inactivity
- **Net effect:** reduced latency and lower server overhead versus non-persistent connections



Discussion Question:

If persistent HTTP fetches eleven sequential objects over one connection without pipelining, what latency is avoided relative to eleven separate connections?

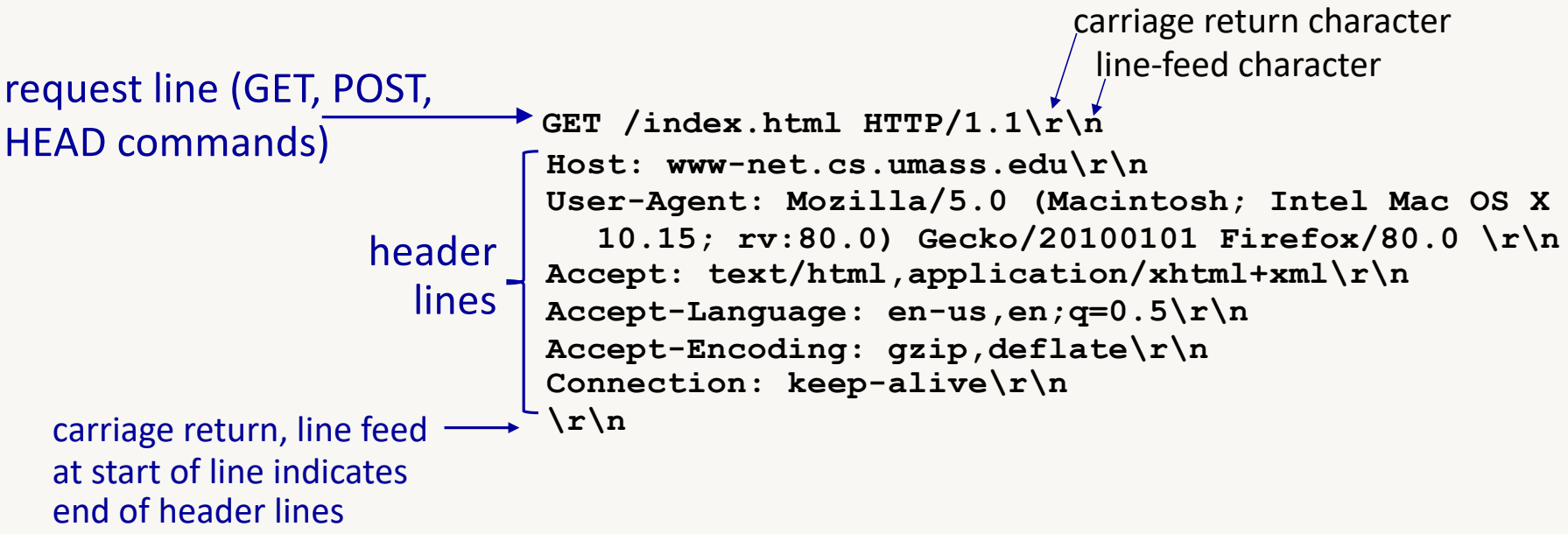
Only the first object requires the TCP setup RTT, so ten additional setup RTTs are avoided. Each sequential request-response exchange can still add an RTT, so persistence does not eliminate all per-object latency.



HTTP/1.1 request message

HTTP request message:

- ASCII (human-readable format)

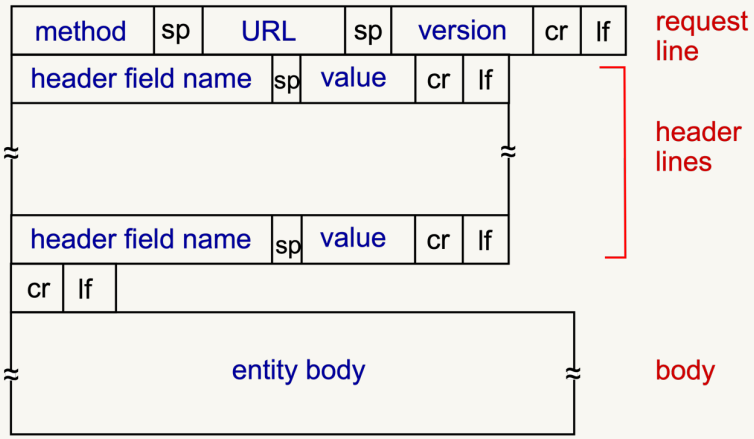


HTTP Message Format



HTTP Request Message:

- **Common methods: GET** (fetch), **POST** (send form/data in body), **HEAD** (headers only), **PUT** (upload/replace), **DELETE** (remove)
- **Headers (examples):**
  - **Host:** target virtual host (required; supports proxies/virtual hosting)
  - **Connection:** e.g., close to avoid persistence
  - **User-Agent:** client type; servers may tailor content
  - **Accept-Language:** enables content negotiation (e.g., prefer French)
- **Entity body:** empty with **GET**; carries data with **POST** (e.g., form fields)

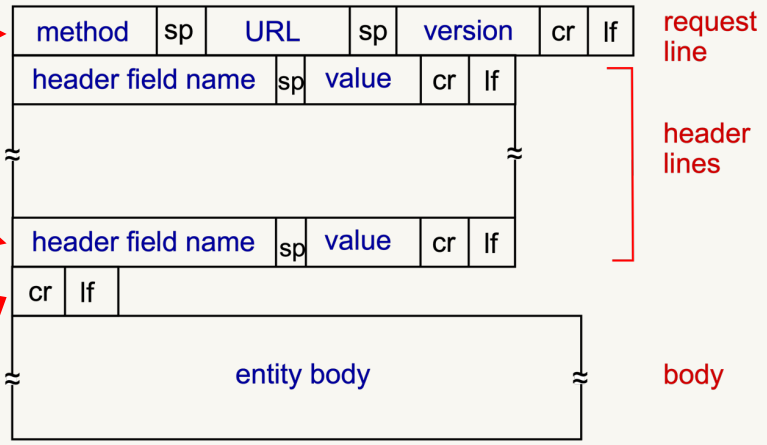


HTTP Message Format



HTTP Request Message:

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac
   OS X 10.15; rv:80.0) Gecko/20100101
   Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```



Other HTTP Request Messages



- **GET**
  - Retrieves a resource and may include parameters in the URL
  - Example: `animalsearch?animal=monkey&food=banana`
- **POST**
  - Submits form or other data in the request body
  - The server processes the submitted data
- **HEAD**
  - Returns the same headers as GET, but without the response body
  - Useful for checking metadata or debugging
- **PUT**
  - Sends content in the request body to a specific URL
  - Creates the resource or completely replaces it if it exists

HTTP Response Message



status line (protocol, status code, status phrase)

header lines

data, e.g., requested HTML file

```
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2020 00:53:20 GMT
Server: Apache/2.4.6 (CentOS)
   OpenSSL/1.0.2k-fips PHP/7.4.9
   mod_perl/2.0.11 Perl/v5.16.3
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
ETag: "a5b-52d015789ee9e"
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8
\r\n
data data data data data ...
```

HTTP Response Message



- **Structure**: status line ⇒ header lines ⇒ blank line ⇒ entity body (the requested object)
- **Status line fields**: HTTP version, status code, reason phrase (e.g., “HTTP/1.1 200 OK”)
- **Key response headers (examples):**
  - **Connection**: e.g., close (server will end TCP after this response)
  - **Date**: time the response was generated (not the object’s modification time)
  - **Server**: software generating the response (e.g., Apache)
  - **Last-Modified**: when the object was last changed
  - **Content-Length**: size of the payload in bytes
  - **Content-Type**: type of the payload (e.g., text/html)
  - **Caching-related headers** (e.g., Cache-Control/Expires/ETag) support client/proxy caching

HTTP Response Message

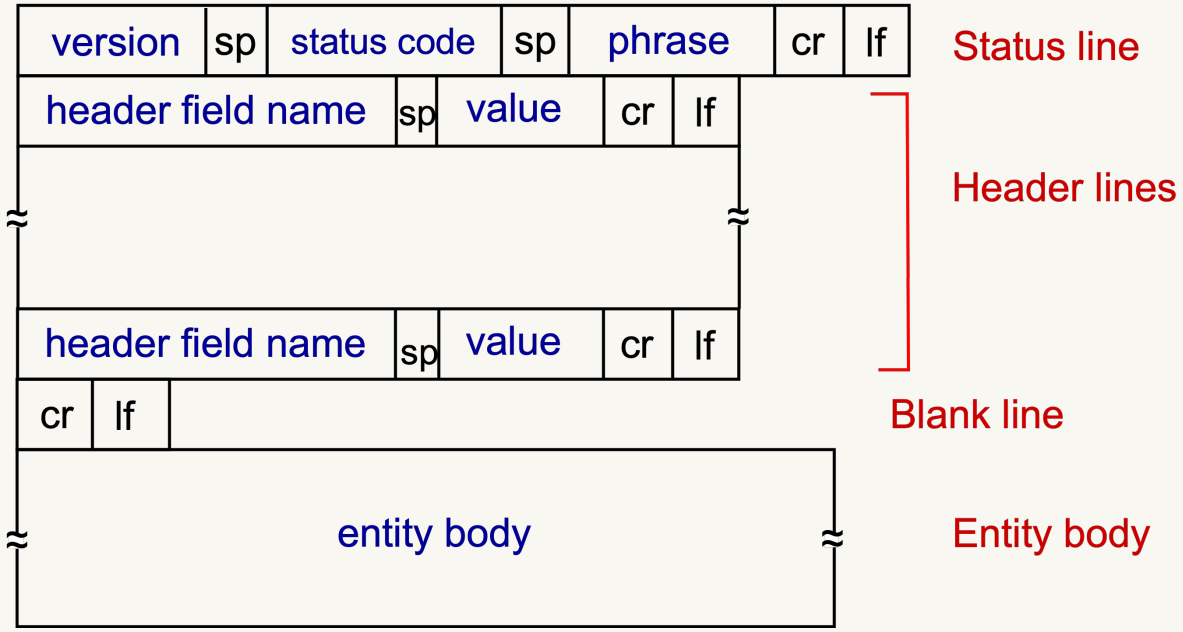


Common status codes:

- **200 OK**: request succeeded; content returned
- **301 Moved Permanently**: Requested object has been permanently moved; the new URL is specified in Location: header of the response message
- **400 Bad Request**: This is a generic error code indicating that the request could not be understood by the server
- **404 Not Found**: The requested document does not exist on this server
- **505 HTTP Version Not Supported**: The requested HTTP protocol version is not supported by the server

HTTP Response Message





Trying Out an HTTP Client



1. netcat to your favorite Web server:

```
% nc -c -v info.cern.ch 80 (for Mac)
>ncat -C info.cern.ch 80 (for Windows)
```

- opens TCP connection to port 80 (default HTTP server port) at `info.cern.ch`
- anything typed in will be sent to port 80 at `info.cern.ch`

2. type in a GET HTTP request:

```
GET /hypertext/DataSources/WWW/Servers.html HTTP/1.1
Host: info.cern.ch
```

- After typing this, hit carriage return twice. You send this GET request to HTTP server

3. Check the response message sent by HTTP server!

(use Wireshark to look at captured HTTP request/response)

HTTP/2



HTTP/1.1

- Supports multiple pipelined requests over one TCP connection
- Responses must follow request order
- **Head-of-line blocking**: small objects may wait behind large ones
- TCP packet loss can delay all objects on the connection

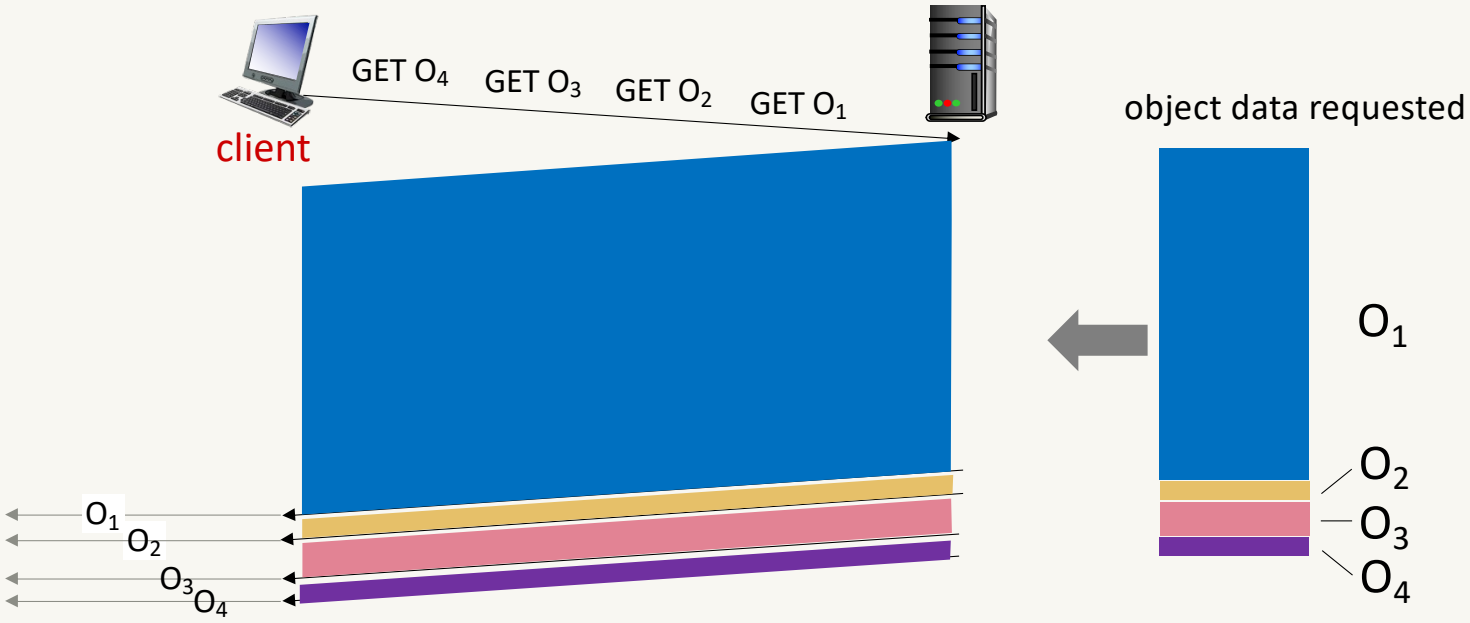
HTTP/2

- Retains HTTP methods, status codes, and most headers
- Splits objects into frames and multiplexes them over one connection
- Uses priorities to influence delivery order
- Originally introduced **Server Push** for sending related resources proactively
- Interleaving frames allows small objects to arrive without waiting for a large object to finish

HTTP/1.1: HOL Blocking



HTTP/1.1: client requests 1 large object (e.g., video file) and 3 smaller objects

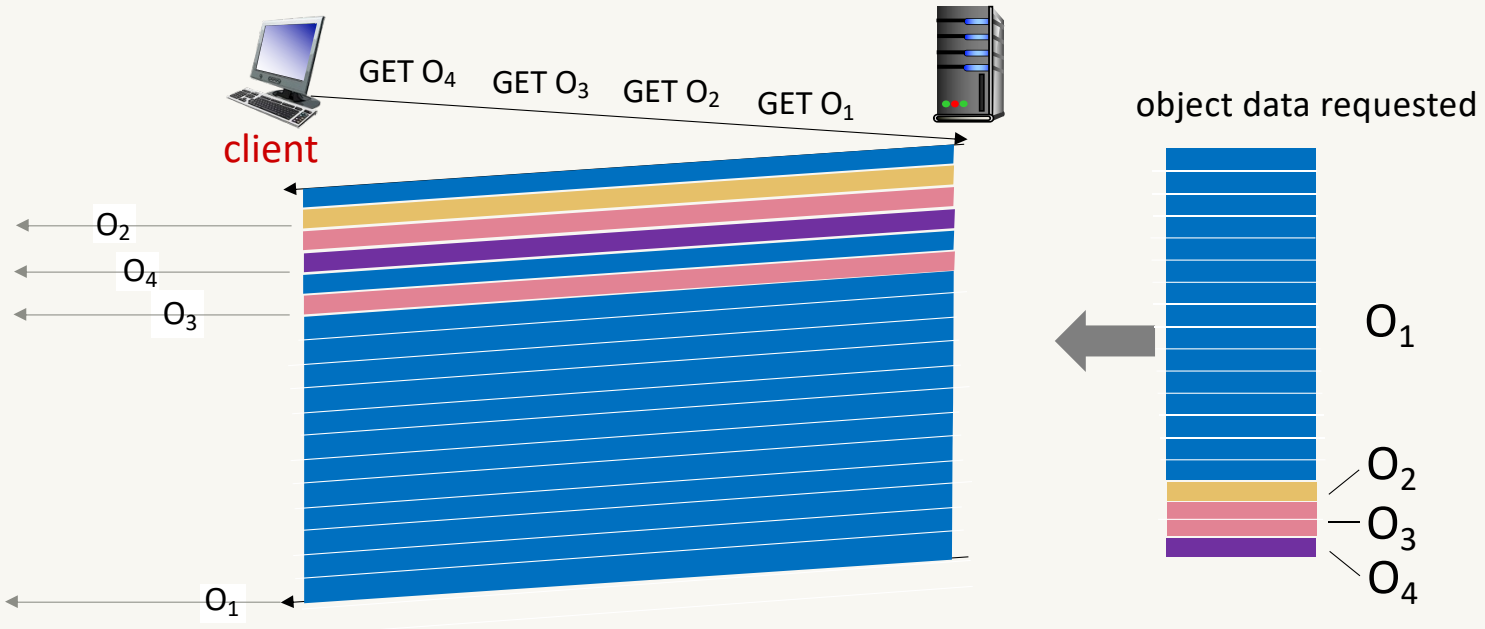


objects delivered in order requested: O_2, O_3, O_4 wait behind O_1

HTTP/2: Reducing HOL Blocking



HTTP/2: objects divided into frames, frame transmission interleaved



O_2, O_3, O_4 delivered quickly, O_1 slightly delayed

HTTP/2 to HTTP/3

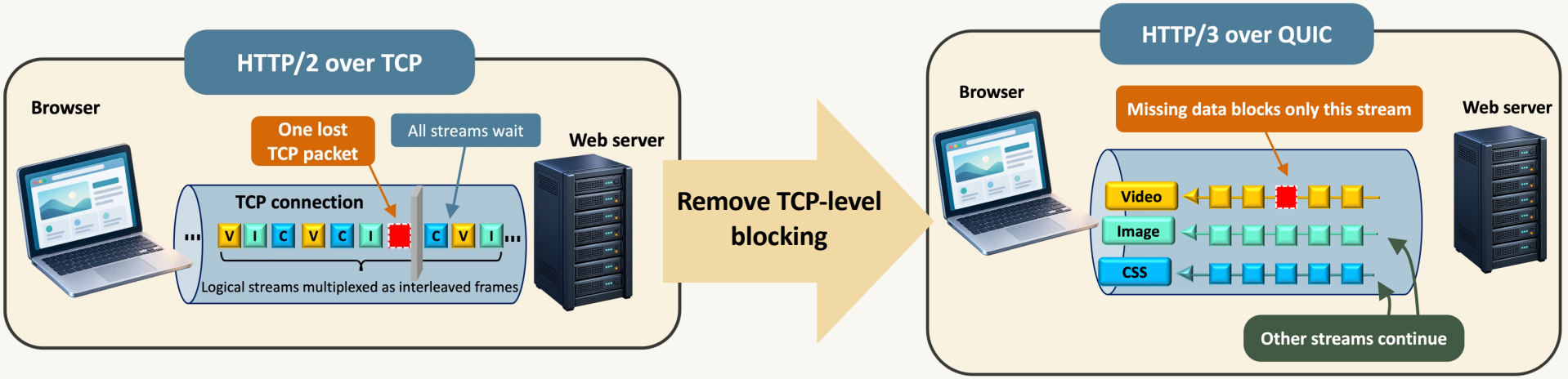


HTTP/2 over TCP

- Multiple HTTP/2 streams share one TCP connection
- A missing TCP segment can temporarily stall delivery to all streams
- HTTP/2 commonly uses TLS over TCP

HTTP/3 over QUIC

- Runs over QUIC carried in UDP
- Integrates TLS 1.3 security
- Uses independent streams and per-stream loss recovery
- Missing data blocks only the affected stream
- Congestion control remains connection-wide





Discussion Question:

Why can prioritizing a small object in HTTP/2 influence normal transmission order but not bypass missing TCP bytes?

Priority guides which available frames the HTTP/2 sender transmits first. Once TCP bytes are missing, the receiver must wait for ordered recovery regardless of the HTTP/2 priority assigned to later frames.

QUIC: Quick UDP Internet Connections

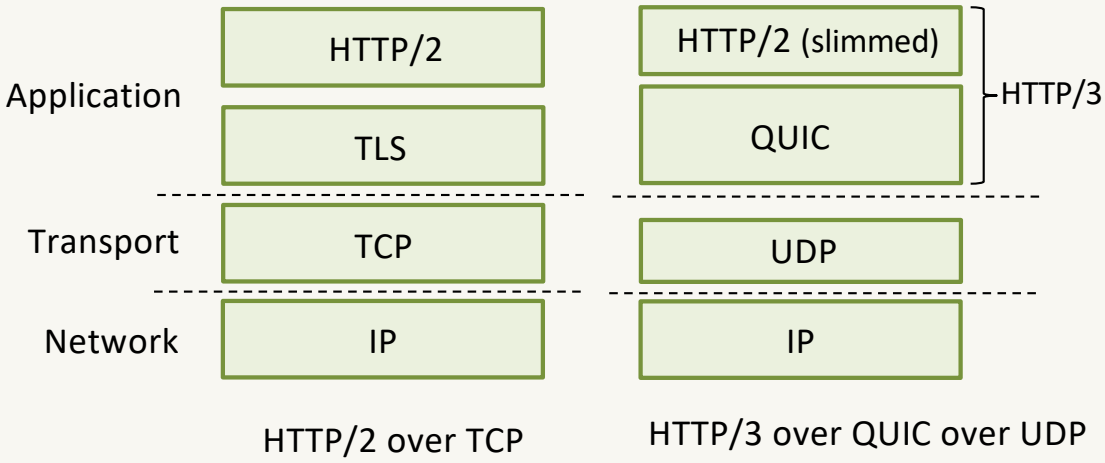


- HTTP/3 uses QUIC, the Quick UDP Internet Connections protocol
- Despite QUIC's role, the Internet still has two transport protocols: TCP and UDP
- QUIC is not a transport-layer protocol; it is an application-layer sublayer that uses UDP to send/receive packets
- Called Quick UDP Internet Connections (runs over UDP, not TCP)

QUIC: Quick UDP Internet Connections

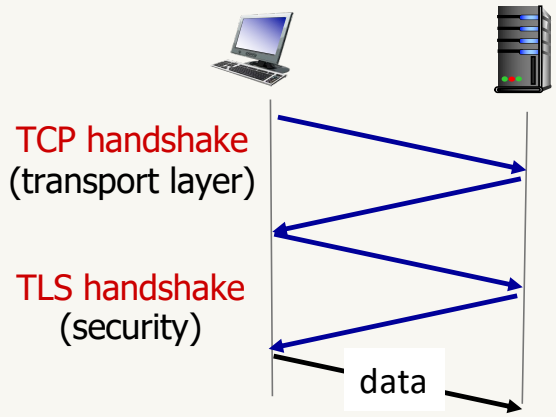


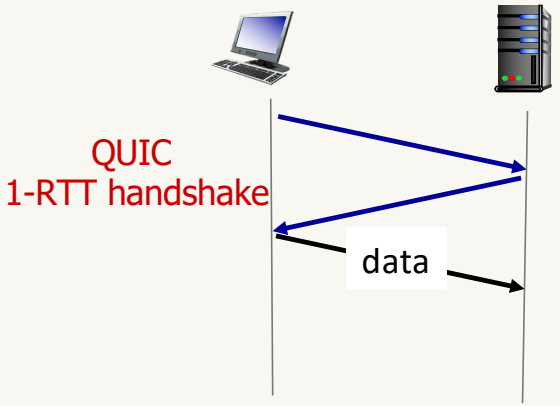
- **Loss & congestion control**: QUIC's mechanisms closely mirror TCP's well-known algorithms
- **Connection establishment**: In a single RTT, QUIC sets up reliability and congestion-control state and establishes authenticated, encrypted connection context
- **Built-in security**: integrates TLS (encryption, authentication, integrity) directly, doesn't need TLS-over-TCP handshake



QUIC: Connection Establishment







TCP (reliability, congestion control state) + TLS (authentication, crypto state)

- 2 serial handshakes

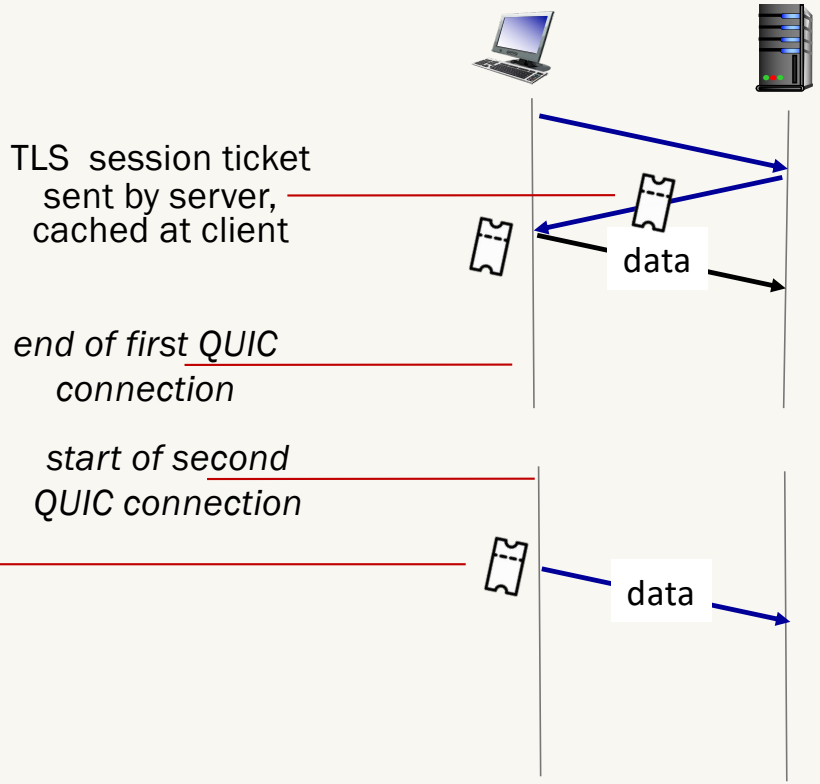
QUIC: reliability, congestion control, authentication, crypto state

- 1 handshake

QUIC: 0-RTT Connection Resumption



0-RTT resumption lets returning clients send data immediately



Client uses cached session ticket for encryption, authentication for new connection

- 0-RTT handshake delay

References



- Ch. 2, Kurose and Ross



Questions?